Docker

Good for engineering

Docker:

-Virtualization software, making developing and deploying application much easier

-Packages application with all the necessary dependencies, configuration, system tools and runtime.

-Portable artifact, easily shared and distributed

Before docker:

- Each dev needs to install and configure all services directly on their os on their local machine
- depend on the different os, installation process different
- Maybe steps, where something can go wrong

With container: Standardize the process of running any service on the local machine

- Own isolated environment
- Postgres packaged with all dependencies and configs
- Start service as a docker container using a 1 docker command
- Command same for all OS
- Command same for all services
- Easy to run different versions of same app without any conflicts

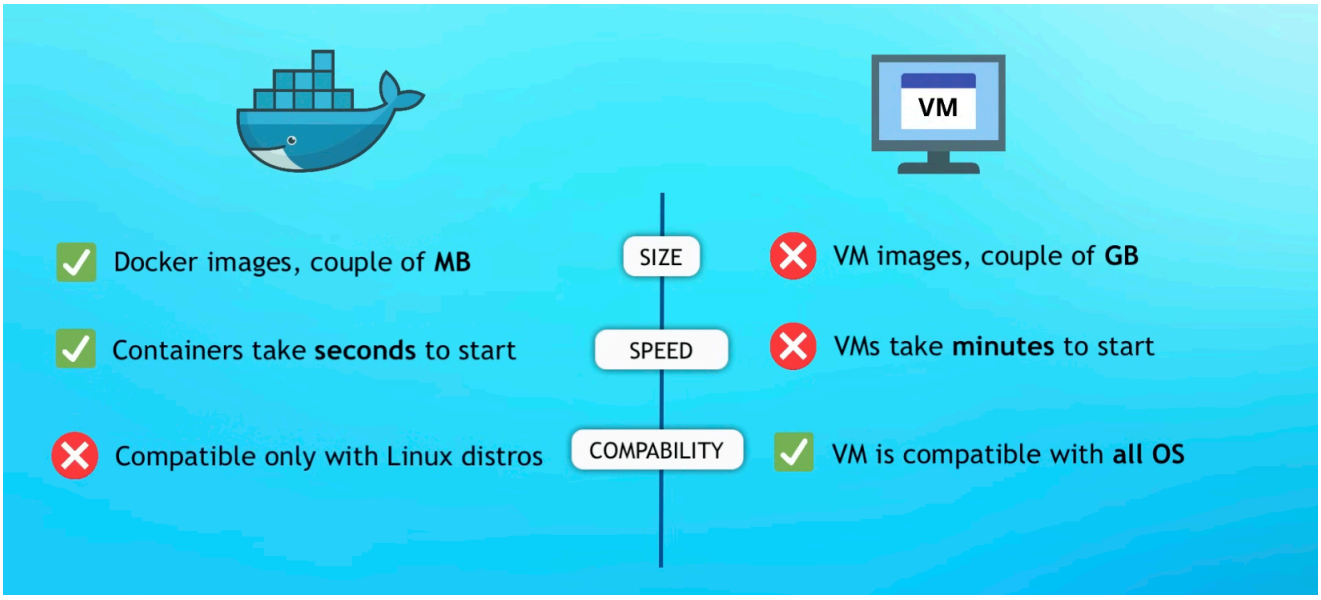
Virtual Machine vs Docker

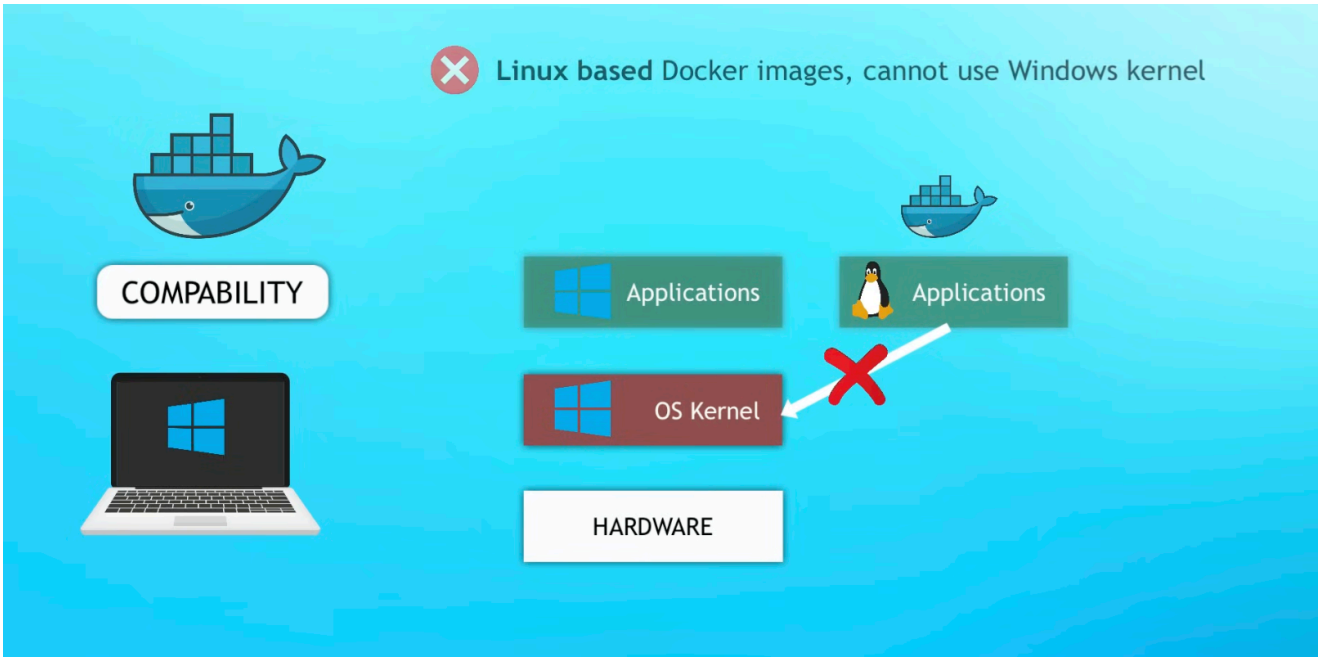
How an OS is made up

The os has two main layer:

- OS application layer: apps
- OS kernel:
    - Kernel is at core of every OS
    - Kernel interacts between hardware and software components

The container work on only the first layer





But, most containers are linux based.
and docker are built for linux

Docker desktop = Allows you to run Linux containers on Windows or MacOS
Because it uses a Hypervisor layer with a lightweight Linux distro

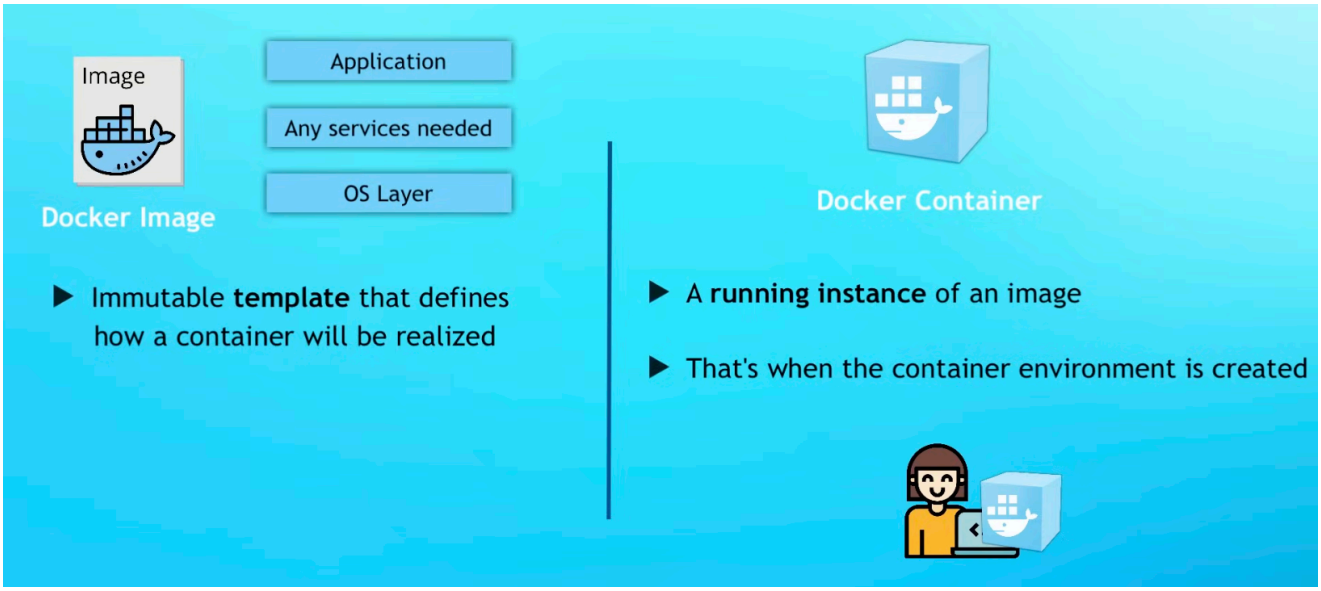
Docker Image Vs Docker Containers

Docker Image:

- An executable application artifact
- Include app source code, but also complete environment configuration.
- Add n variables, create directories, file, etc

Docker Container

- Actually starts the application



And you can run multiple containers from 1 image.

Command line:
`- docker images` - docker ps = List running containers `

Docker Registries

- A storage and distribution system for Docker Images
- Official images available from applications like Redis, Mongo, Postgres, etc.
- Official images are maintained by the software authors or in collaboration with the docker community

And Docker hosts one of the biggest Docker Registry, called "Docker Hub"

Docker Official Images

- A dedicated team responsible for reviewing and publishing all content in the Docker Official Images repositories
- Works in collaboration with software maintainers, security experts
- However, anyone can participate as collaboration takes place only on Github

```
docker run nginx
docker run nginx
docker run nginx
```

This creates 3 separate containers from the same image, meaning that one image can create many containers.

Docker pull
`docker pull {name}:{tag}` = Pull an image from a registry

Docker run
`docker run {name}:{tag}` = Creates a container from given image and starts it

`docker run -d {name}:{tag}` = detach it from the terminal = run container in background and prints the container ID

but you may still want to see the logs, which can be useful for debugging etc.

Docker logs
`docker logs {container}` = view logs from service running inside the container. (which are present at the time of execution)

Note: any images that we don't have locally, we can still run directly using the docker run command.
So the what it does is that

- first it locates the image in the local
- if not, it will locate that image on the docker hub and pull the image from there

Port Binding

Port binding = Bind the container's port to the host's port to make the service available to the outside world.

`docker run -d -p {HOST_PORT}{CONTAINER_PORT} {name}:{tag}` = Publish a container's port to the host

`docker ps -a or --all` = lists all containers (stopped and running)

`docker stop {Container}`

`docker run --name {name} -d -p 9000:80 nginx:1.23` = assign a name to the container

To check
`docker log {name}`

Building own Docker Images

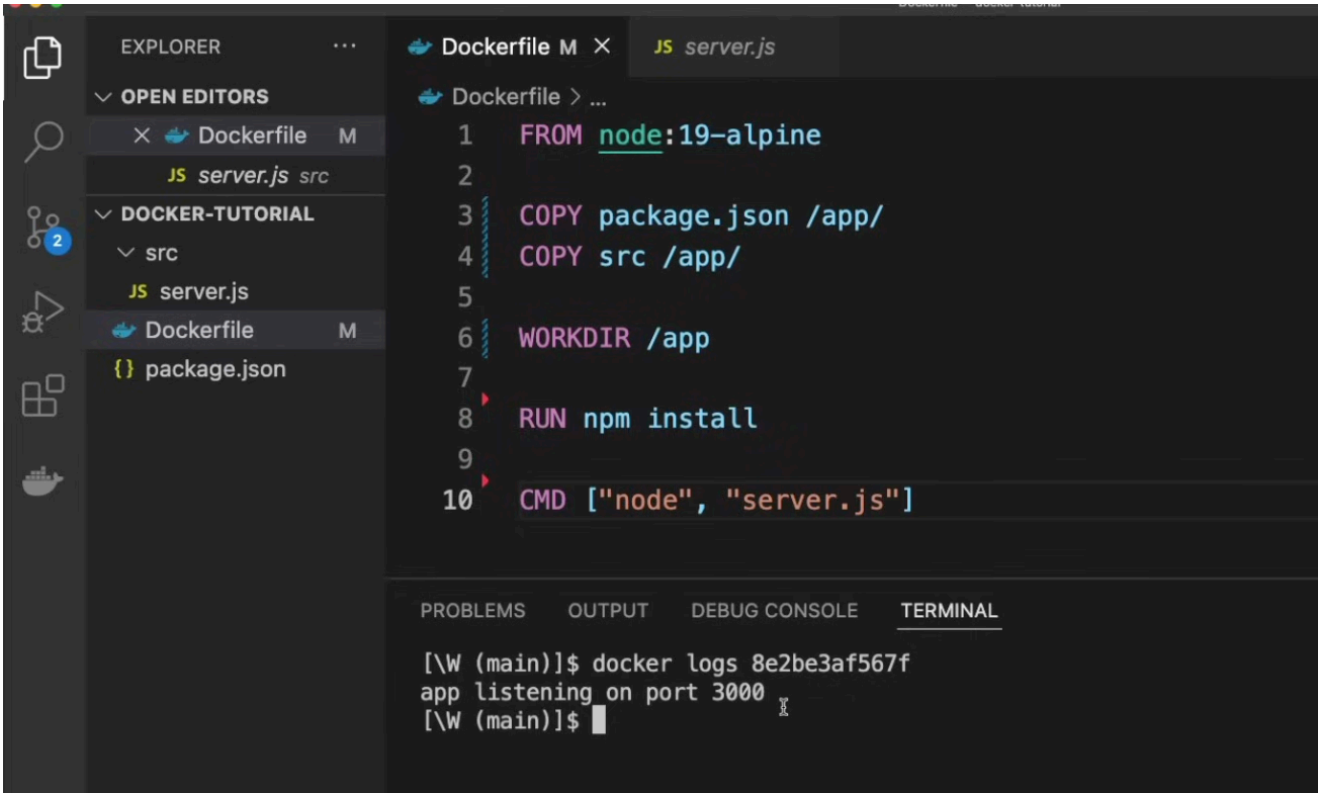
- Deploy to the server
  we want to deploy our app as a Docker container
- Dockerfile is a text document that contains commands to assemble an image
- Docker can then build an image by reading those instructions

Structure of Dockerfile

- Dockerfiles start from a parent image or 'base image'
- It is a Docker Image that your image is based on

`You choose the base image, depending on which tools you need to have available.`

```
Dockerfile
16 FROM python:3.12-slim
15
14 WORKDIR /app
13
12 COPY requirements.txt .
11
10 RUN pip3 install --no-cache-dir -r requirements.txt
 9
 8 COPY . .
 7
 6 EXPOSE 5000
 5
 4 ENV DEBUG=False
 3 ENV PORT=5000
 2 ENV LOG_FILE=/data/logs/logs.txt
 1
17 CMD ["python3", "app.py"]
```



`CMD` = used to execute a specific command.

Note: It is not executed during build but when the container is created or started

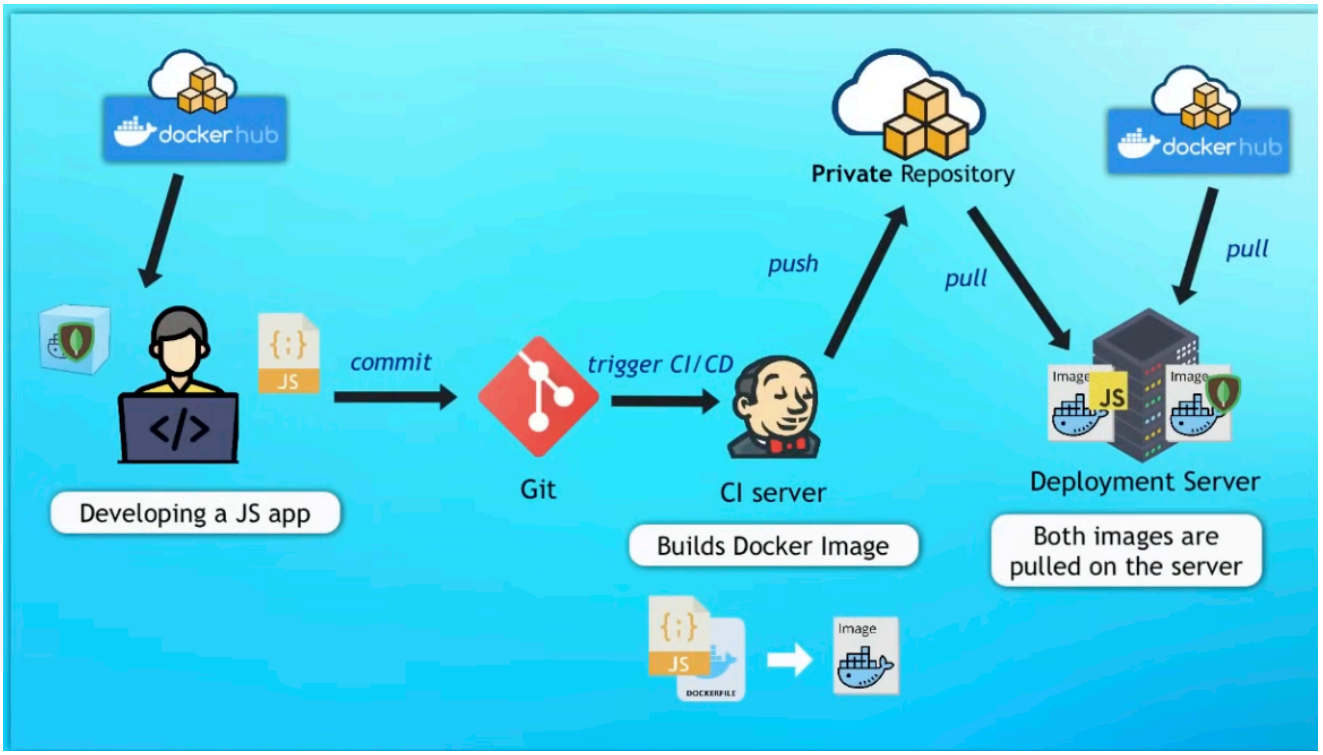
`WORKDIR`

- sets the working directory for all following commands
- Like changing into a directory: 'cd ...'

Build Image

Remember: A Docker image consists of layers

- Each instruction in the Dockerfile creates one layer
- These layers are stacked & each one is a delta of the changes from the previous layer.



`docker build --tag {name} .` = this is going to create an image that we can use to build a container and run the container.

Process

After the DockerFile has been created
Next objective to defy is

- **Build image**
  `docker build -t my-python-app .`
- **Run the container**
  `docker run -p 8888:8888 -p 5000:5000 my-python-app`

Jupyter notebook

`docker run -p 8888:8888 -v $(pwd):/app cardiovascular-classifier:latest`